

PROYECTO LOONA · STUDIO OS · AGOSTO 2026

Cómo construir tu propio Jarvis: instrucciones claras, repos reales y cero bait de DM

Guía operativa para Chuy. Un asistente personal siempre encendido, con voz, memoria, herramientas y guardrails — no un chatbot con nombre fancy.

Campo	Valor
Nombre del agente	LOONA
Workspace herdr	LOONA (wB) · /Users/imac/loona
Máquina de trabajo	iMac Intel Core i5-6500 · 24 GB RAM · macOS 12.7.6 Monterey
Ya instalado	Python 3.14 · Node 24.18 · Ollama 0.32 · herdr 0.8
Método de investigación	X (Latest, 6–12 ago 2026) + README oficiales de cada repo
Filtro	Se descartaron tweets que piden DM / “comenta JARVIS” / pack privado
Recomendación principal	OpenClaw (cerebro 24/7) + Personal Jarvis (voz y escritorio)

Este PDF no es un resumen de hype. Cada paso se puede ejecutar hoy en esta máquina. Cuando un stack exige Apple Silicon o GPU NVIDIA, se marca como opcional o futuro.

Contenido

1. Introducción — qué es un Jarvis de verdad
2. Radar de X (agosto 2026) — repos públicos, sin DM
3. Equipo y capacidades (hardware)
4. Software, internet e IA
5. Arquitectura recomendada para LOONA
6. Paso a paso A — OpenClaw, el cerebro siempre encendido
7. Paso a paso B — Personal Jarvis, voz y control del Mac
8. Paso a paso C — voz local (Hugging Face speech-to-speech)
9. Identidad, memoria y guardrails de LOONA
10. Primeras 24 horas — checklist de prueba
11. Seguridad, costos y anti-patronos
12. Fuentes y repos canónicos

1. Introducción — qué es un Jarvis de verdad

En X, “construye tu Jarvis” se volvió un género. La mayoría de esos posts no entregan un asistente: entregan un chatbot con wake word, o peor, un hilo que termina con “comenta JARVIS y te lo mando por DM”. Eso no es un proyecto. Es un lead magnet.

Un Jarvis real no es un modelo. Es un sistema de cinco capas que trabaja por ti mientras no estás mirando la pantalla. Fluixoo lo resumió bien el 12 de agosto de 2026: identidad, memoria, herramientas, triggers y guardrails. Sin herramientas solo habla. Sin memoria empieza de cero. Sin triggers tú sigues haciendo el trabajo. Sin guardrails es un riesgo.

“Everyone wants Jarvis. Most people build a chatbot with a cool name. That is not the same thing.” —
@fluixoo, 12 ago 2026

Las cinco capas (criterio de diseño de LOONA)

Capa	Pregunta que responde	Si falta
Identidad	¿Quién es LOONA, para quién trabaja, con qué tono, qué nunca hace?	Suena genérico. Mezcla proyectos. No tiene criterio.
Memoria	¿Qué debe recordar entre sesiones? Preferencias, gente, repos, vetos.	Cada mañana es un extraño. Repites el mismo briefing.
Herramientas	¿Puede ejecutar? Archivos, terminal, calendario, browser, WhatsApp, código.	Es un chat. No un asistente.
Triggers	¿Actúa sola? Cron, wake word, mensaje de Telegram, evento de calendario.	Tú sigues empujando cada tarea.
Guardrails	¿Qué requiere tu sí? Pagos, publish, borrar, DM a terceros, keys.	Un error caro o un leak de secretos.

Definición de done de este proyecto: LOONA responde por voz o por Telegram, recuerda algo que le dijiste ayer, ejecuta al menos una herramienta real (archivo, comando o canal) y pide aprobación antes de cualquier acción irreversible.

Por qué este PDF no te manda a “vibe-code Jarvis en 20 minutos”

Riley Brown (@rileybrown) tiene un tutorial viral de julio 2026: Cursor + GPT-Realtime-2 y un prompt largo. Es útil como inspiración de tools, pero no es un repo configurable que puedas versionar, auditar y reabrir en un mes. LOONA necesita código público, config en disco y un camino de update. Por eso priorizamos repos MIT con installer y archivo de configuración, no hilos de “pégame este prompt”.

2. Radar de X (agosto 2026) — repos públicos, sin DM

Se buscaron tweets recientes (modo Latest) con combinaciones de “build your own Jarvis”, “personal AI assistant”, “openclaw”, “github.com” y equivalentes en español. Se excluyeron posts cuyo call-to-action es mandar DM, comentar una palabra o pedir un pack privado. Se priorizaron los que ya publican el enlace del repositorio.

Repos que sí pasaron el filtro

Repo	Quién lo empuja en X	Por qué importa para LOONA	Veredicto
openclaw/openclaw docs.openclaw.ai	@BuiltByNav, @sanjibxai, @YasKad4 (9–12 ago). ~385k stars.	Asistente self-hosted 24/7. Gateway + canales (Telegram, WhatsApp, Slack, iMessage). Skills y plugins. MIT. Configurable de verdad.	CEREBRO BASE

Repo	Quién lo empuja en X	Por qué importa para LOONA	Veredicto
PersonalJarvis / PersonalJarvis v1.3.0 · MIT	@PersonalJarvis (10–12 ago). Mac / Windows / Linux.	Voz, dictado, computer-use, wiki Markdown, wake word propio, Ollama local, Claude/Codex/Gemini como workers. Un comando de install.	CAPA VOZ / DESKTOP
huggingface / speech-to-speech ~11.6k stars	@Marco_Ramilli (8 ago). “Want to build your own Jarvis?”	Pipeline VAD → STT → LLM → TTS, API compatible con OpenAI Realtime. Local o híbrido. Piezas intercambiables.	Voz local avanzada
MOVI85/flightdeck MIT	@Slevin_Fi (26 jul). Wake word “Jarvis”, 80+ skills.	HUD cinematográfico + Hermes Agent + Whisper local. El más “Iron Man”. Requiere macOS Apple Silicon para la mitad ambient.	Futuro (no este iMac)
memovai/mimiclaw	@tom_doerr (10 ago). 21 likes, 25 bookmarks.	ESP32-S3 de ~5 USD + Telegram. Jarvis de hardware barato, no de escritorio.	Experimento hardware
garrytan/gbrain	@SharavArora, @edsoehnel (abr 2026). CEO de YC.	Segundo cerebro persistente: meetings, mail, ideas, evidencia. Complemento de memoria, no el orquestador.	Memoria avanzada (fase 2)

Tabla 1. Fuentes de X verificadas contra el README de cada repo el 12 de agosto de 2026.

Menciones útiles pero incompletas

- @fluixoo (12 ago): el marco de 5 capas. Sin repo. Se adopta como doctrina, no como código.
- @HernandezFreddy (12 ago): Mac + Firecrawl + OpenClaw. Video corto, sin repo propio; confirma el stack OpenClaw.
- @rileybrown (1 jul): tutorial Cursor + GPT-Realtime-2, 1.3k bookmarks. Buen patrón de tools, mal como base versionable.
- @nikhilachale/GWEN: voz + Claude + ElevenLabs + macOS. Repo real, pero menos maduro que Personal Jarvis.
- @tominaga-h/jarvis-shell: shell AI-nativo en Rust. Interesante, no es un asistente de vida.
- iboss21/free-jarvis-template (REGES): tweet del 11 ago. Wizard configurable, pero Windows-first y voz “needs binaries”. No es el camino de este iMac.

Lo que se descartó a propósito

Patrón en X	Por qué se tira	Ejemplo de esta semana
“Comenta JARVIS / drop a ■ / DM me la palabra”	El código no es público. No se puede auditar, clonar ni actualizar. Suele ser un funnel.	Cientos de hilos de “build your Jarvis” que terminan en inbox.
Repo de 1 archivo Python “voice assistant”	speech_recognition + webbrowser.open. Demo de facultad, no sistema.	@AyaanAli___ / @Naveen_RoyalIII — válidos como hobby, no como LOONA.
Pack privado en GitHub privado	Si el autor dice “está en GitHub pero lo tengo private, DM me”, no es configurable para ti.	@SteviKelly, 12 ago: packet de founder command, repo privado.
Flightdeck / AIRI como día 1	Espectacular, pero asume Apple Silicon, Hermes, ElevenLabs y paciencia de permisos.	@Stefan_3D_AI (AIRI) y @Slevin_Fi (Flightdeck).

3. Equipo y capacidades (hardware)

Un Jarvis no pide el mismo hierro si el cerebro vive en la nube o en tu disco. Esta sección separa lo mínimo para arrancar hoy, lo recomendado para voz cómoda, y lo que este iMac sí / no puede hacer.

Tu máquina actual (medida el 12 ago 2026)

Componente	Lo que hay	Implicación para Jarvis
CPU	Intel Core i5-6500 @ 3.20 GHz (4 núcleos, 2015–16)	Sirve para orquestar. No sirve para modelos locales grandes con latencia de conversación.
RAM	24 GB	Holgada para OpenClaw + browser + herdr. Justa para un modelo local 7B cuantizado + STT.
GPU	Intel HD 530 integrada (sin CUDA, sin Apple Neural Engine)	Whisper/TTS locales serán lentos. mlx-audio y Flightdeck nativo no aplican.
OS	macOS 12.7.6 Monterey	OpenClaw y Personal Jarvis declaran soporte Mac. Evita stacks que piden Sequoia + Swift reciente.
Audio	Micrófono y parlantes del iMac	Suficiente para el día 1. Un mic USB con mute físico es el primer upgrade barato.

Tres niveles de equipo

Nivel	Equipo	Qué Jarvis te da	Costo marginal
A — Arranque (este iMac)	iMac actual + mic + internet estable + (opcional) iPhone para Telegram	Cerebro 24/7 en OpenClaw, voz de escritorio con Personal Jarvis, modelos cloud (Grok/Claude/Gemini). Local solo para cosas chicas (Ollama 3B–7B).	\$0. Ya lo tienes.
B — Voz cómoda	Mic USB (Jabra/Elgato Wave) + audífonos + UPS si lo dejas 24/7 + disco SSD libre ≥ 40 GB	Menos errores de wake word, dictado usable, modelos STT/TTS cacheados sin llenar el disco.	USD 40–150
C — Jarvis local de verdad	Apple Silicon 32 GB+ (M-series) o PC/NVIDIA 12 GB VRAM. O un MiniPC/NUC siempre encendido.	Speech-to-speech local, Flightdeck completo, 14B–32B en casa, privacidad máxima.	USD 800–2 500. No es el día 1.

Regla honesta para este iMac

No intentes “todo local” el primer fin de semana. El cuello de botella no es la RAM: es la CPU de 2015 y la GPU integrada. LOONA debe pensar en la nube (Grok / Claude / Gemini, que ya pagas) y usar el Mac como orquestador, micrófono y ejecutor de tools. Ollama se reserva para pruebas privadas y fallback, no para la voz en tiempo real.

Periféricos y “capacidades” que sí importan

- **Micrófono:** el del iMac sirve. Si hay eco de los parlantes, usa audífonos. El wake word odia el playback.
- **Red:** 20 Mbps simétricos bastan. Lo crítico es latencia estable, no el gigabit. Evita Wi-Fi saturado si usas voz cloud.
- **Siempre encendido:** un Jarvis de verdad no se apaga. Deja el iMac despierto o mueve OpenClaw a un MiniPC/VPS más adelante.
- **Teléfono:** Telegram o WhatsApp son el “HUD de bolsillo”. No necesitas app nativa el día 1.
- **Disco:** reserva 15–40 GB para modelos Whisper/Piper/Ollama. No los descargues todos.
- **Cámara:** opcional. Solo si más tarde quieres “qué hay en mi pantalla” o computer-use con visión.

4. Software, internet e IA

Ya lo tienes (no instales de más)

Pieza	Versión vista	Rol en LOONA
herdr	0.8.0	Workspace LOONA. Panes para OpenClaw, Personal Jarvis, logs.
Node.js	24.18.0	Runtime de OpenClaw (pide 22.22.3+ / 24.15+ / 25.9+). Perfecto.
Python	3.14.6	Personal Jarvis pide 3.11+. Si 3.14 rompe wheels, usa 3.12 vía pyenv/brew.
Ollama	0.32.0 (cliente)	Cerebro local opcional. Hoy no hay daemon corriendo: hay que levantarlo.
Grok / SuperGrok	pagado	Cerebro fuerte para razonar, orquestar y verificar.
Claude + Codex	panes herdr	Workers de misiones largas (código, docs).
Git + navegador	sistema	Clonar repos, dashboard de OpenClaw, HUD.

Qué hay que instalar

Software	Para qué	Obligatorio día 1?	Costo
OpenClaw (npm / install.sh)	Gateway 24/7, canales, skills	Sí	Gratis (MIT). Pagas el modelo.
Personal Jarvis	Voz, dictado, mouse/teclado, wiki	Sí, si quieres hablarle al iMac	Gratis (MIT).
Git (si faltara)	Clonar y versionar LOONA	Sí	Gratis
Ollama + qwen2.5:3b o 7b	Fallback local / privacidad	Recomendado	Gratis
Telegram BotFather	Canal de bolsillo de LOONA	Recomendado	Gratis
Cuenta xAI / Anthropic / Google / OpenAI	Cerebro cloud	Sí, una basta	Ya pagado o pay-per-token
ElevenLabs / Edge-TTS / Piper	Voz hablada	Opcional	Piper/edge-tts = \$0
Firecrawl / Exa	Web search estructurado	Fase 2	Free tier o API
Obsidian	Leer la wiki de memoria en humano	Opcional	Gratis
Docker	Solo si quieres speech-to-speech aislado	No	Gratis

Cuentas e internet (mapa de APIs)

Servicio	Se usa para	Dónde vive la key	Regla
xAI / Grok	Razonamiento, verify, copy	Keychain / .env fuera de git	Preferido del studio
Anthropic Claude	Misiones largas, código	Igual	Worker, no default 24/7 si el costo pica
Google Gemini	Barato y multimodal	Igual	Buen default de OpenClaw
OpenAI Realtime	Voz sub-segundo	Igual	Opcional. Caro si dejas el mic abierto
Telegram Bot API	Hablarle a LOONA del celular	openclaw config	El canal más rápido de setupear
WhatsApp (OpenClaw)	Canal de vida real	pairing + número	Fase 2. Más fricción que Telegram
Ollama local	Datos que no quieres subir	localhost:11434	Sin key

Nunca pegues API keys en un tweet, en un prompt de agente público, ni en el repo. Personal Jarvis las guarda en el llavero del sistema. OpenClaw usa su propio state dir (~/.openclaw). Añade .env y jarvis.toml con secretos al .gitignore

desde el minuto uno.

5. Arquitectura recomendada para LOONA

No instales cuatro Jarvis a la vez. Eso es el anti-patrón de la banda: cuatro agentes peleando el mismo write-path. LOONA tiene un solo nombre, dos procesos, un contrato.

```
TU (voz / Telegram / herdr)
|
+-- Personal Jarvis -> oye, habla, dicta, mouse, wiki local
| |
| +-- (tareas pesadas) Claude Code / Codex / Gemini CLI
|
+-- OpenClaw Gateway -> 24/7, skills, cron, canales
|
+-- Cerebro: Grok o Gemini (cloud) + Ollama (fallback)
+-- Memoria: Markdown (~/.openclaw + wiki LOONA)
+-- Canales: Telegram ahora / WhatsApp despues
+-- Tools: exec, browser, files, calendar, scripts
```

Por qué este corte (y no Flightdeck hoy)

- **OpenClaw** es lo que X llama “el Jarvis que de verdad existe” en 2026: self-hosted, MIT, canales, skills, un Gateway. Encaja con Node 24 que ya tienes.
- **Personal Jarvis** es el único desktop voice stack maduro, multi-OS, con installer de un comando, wake word configurable (la pondremos en “Loona”) y computer-use.
- **Flightdeck** es el más bonito, pero sus apps nativas (wake, orb, gestures) son Swift/macOS Apple Silicon. En este iMac Intel perderías la mitad ambient.
- **speech-to-speech** se añade cuando la voz cloud moleste o quieras offline. No bloquea el día 1.

Contrato de la banda (founder-studio)

Un write-path por proceso. OpenClaw es dueño de canales y cron. Personal Jarvis es dueño del micrófono y del escritorio. Grok verifica. Nadie declara “ya quedó” sin una prueba: un mensaje respondido, un archivo creado, o una URL / log. Free-first: Piper/edge-tts y Ollama antes de ElevenLabs o un VPS nuevo.

6. Paso a paso A — OpenClaw, el cerebro siempre encendido

Tiempo estimado: 15–25 minutos. Resultado: un Gateway en el puerto 18789, un dashboard en el browser y LOONA contestando en el Control UI. Después, Telegram en el celular.

PASO A1 — Abre el workspace correcto

El space herdr **LOONA** ya está creado (id wB) con cwd /Users/imac/loona. Trabaja ahí, no en likinya. En la TUI: click en LOONA, o ctrl+b luego w.

```
cd /Users/imac/loona
herdr workspace list # debe aparecer LOONA focused
node --version # v24.18.0 o superior
```

PASO A2 — Instala OpenClaw

Como ya gestionas Node, puedes ir directo a npm. El installer oficial también instala Node si faltara. Lee el script antes de pipearlo a bash si te da cosa (es open source).

```
# Opción recomendada en esta máquina (Node ya está):
npm install -g openclaw@latest --allow-scripts openclaw
```

```
# Alternativa oficial (macOS):
# curl -fsSL https://openclaw.ai/install.sh | bash

openclaw --version
openclaw doctor
```

Si npm 12 bloquea lifecycle scripts, el flag --allow-scripts openclaw es obligatorio. Documentado en docs.openclaw.ai/install.

PASO A3 — Onboarding (modelo + daemon)

```
openclaw onboard --install-daemon
```

En el wizard, elige en este orden:

- **Nombre del agente:** LOONA (no “Jarvis”, no “Molty”).
- **Provider:** el que ya pagas. Grok/xAI si el conector está; si no, Gemini (barato) o Anthropic.
- **API key:** pégala solo en el wizard. No la dejes en un .txt del Desktop.
- **Daemon:** sí. En macOS instala un LaunchAgent para que sobreviva logout/reboot.
- **Canales:** salta WhatsApp en el primer pase. Telegram viene en A6.
- **Skills extra / plugins:** skip. Vuelves con openclaw configure.

PASO A4 — Verifica el Gateway

```
openclaw gateway status # listening :18789
openclaw dashboard # abre el Control UI
openclaw doctor # debe quedar verde
```

En el Control UI escribe: “Eres LOONA. Confirma tu nombre y dime tres tools que tienes.” Si responde, el cerebro está vivo. Si no, openclaw gateway status y revisa la key del provider.

PASO A5 — Identidad en disco (soul + memoria)

OpenClaw guarda el workspace del agente en su state dir. Crea además un cerebro canónico dentro del repo LOONA para que la banda (Grok, Codex, Claude) lea la misma verdad.

```
mkdir -p /Users/imac/loona/docs /Users/imac/loona/identity
# Crea identity/SOUL.md (quién es, qué hace, qué nunca hace)
# Crea identity/MEMORY.md (hechos estables: zona MTY, studio, proyectos)
# Crea docs/BANDA_CEREBRO_LOONA.md (hecho / falta / sigue)
```

Pégalo en el system / workspace de OpenClaw (el wizard y openclaw configure te dejan apuntar instrucciones). La sección 9 de este PDF trae el texto listo para copiar.

PASO A6 — Telegram (el HUD de bolsillo)

- En Telegram, habla con @BotFather → /newbot → nombre visible “LOONA” → username tipo loona_jarvis_bot.
- Copia el token. Nadie más lo ve.
- En el host: conecta el canal Telegram (docs.openclaw.ai/channels/telegram). Suele ser el canal más rápido.
- Mándale un “hola” desde tu cuenta. OpenClaw hace pairing por defecto con senders desconocidos.
- Aprueba el pairing: openclaw pairing approve telegram <código>.
- Prueba: “recuerda que mi zona horaria es América/Monterrey y el proyecto activo de contenido es Likinya”.

PASO A7 — Primera herramienta útil (no un demo)

Un Jarvis de verdad hace una cosa molesta por ti. Elige UNA, no diez:

- “Lista los workspaces de herdr y dime cuál está focused.”
- “Léeme docs/BANDA_CEREBRO_LOONA.md y dime el P0.”
- “Cada día a las 8:00 MTY, mándame por Telegram las 3 tareas abiertas del studio.” (cron de OpenClaw)
- “Cuando te escriba ‘radar’, corre un resumen de menciones y no publiques nada.”

No le des shell irrestricto el día 1. Lee docs.openclaw.ai/gateway/sandboxing y docs.openclaw.ai/gateway/security antes de exponer el Gateway fuera de localhost. DM-capable channels tratan mensajes inbound como input no confiable. Eso no es paranoia: es el README oficial.

7. Paso a paso B — Personal Jarvis, voz y control del Mac

Tiempo estimado: 20–40 minutos (incluye descargas de modelos de voz). Resultado: una app de escritorio que despierta con la palabra **Loona**, responde en voz alta, dicta en cualquier campo y puede mover mouse/teclado.

Repo canónico: <https://github.com/PersonalJarvis/PersonalJarvis> · MIT · v1.3.0+ · Python 3.11+

PASO B1 — Instala (un comando)

El installer crea un venv, instala deps, precarga modelos de voz y lanza la app. No pide nada en la terminal: el setup ocurre dentro de la app (idioma, wake word, keys).

```
# Lee el script primero si quieres (recomendado):
curl -fsSL https://raw.githubusercontent.com/PersonalJarvis/\
PersonalJarvis/main/install/install.sh | less

# Luego instala:
curl -fsSL https://raw.githubusercontent.com/PersonalJarvis/\
PersonalJarvis/main/install/install.sh | bash
```

Alternativa pipx, aislada: `pipx install personal-jarvis && jarvis serve`. O clone manual en `/Users/imac/loona/vendor/PersonalJarvis` si quieres el código al lado del proyecto.

PASO B2 — Setup dentro de la app

- **Idioma:** auto (español + inglés). LOONA debe espejear el idioma en el que le hablas.
- **Wake word:** vacío por defecto a propósito. Pon **loona**. Nada de “Jarvis” — ese nombre está saturado y se activa con videos de Iron Man.
- **Cerebro (brain tier):** usa una key que ya pagas (Gemini / Claude / OpenAI / OpenRouter). En este iMac no pongas el realtime local experimental (pide ~12 GB de GPU/unified memory).
- **STT:** empieza con un provider cloud barato (Groq/Gemini) para que la latencia no te frustre. Pasa a faster-whisper local solo si te molesta subir audio.
- **TTS:** Gemini Flash TTS o Piper local. Evita ElevenLabs hasta que la voz “barata” te moleste de verdad.
- **Workers:** si ya tienes Claude Code / Codex, conéctalos. Las misiones pesadas salen del router a un worktree aislado y un critic las revisa.

PASO B3 — Config fino (opcional pero correcto)

No necesitas archivo. Si lo quieres versionable (sin secretos), copia el ejemplo oficial:

```
[profile]
language = "auto"

[trigger.wake_word]
phrase = "loona"
engine = "auto"

[stt]
provider = "groq-api" # o gemini-api / faster-whisper

[tts]
provider = "gemini-flash-tts"
fallback = "piper"
```

Overrides por ENV: `JARVIS__SECTION__KEY=...` · Las keys NUNCA van en este toml; viven en el llavero o en `.env`.

PASO B4 — Permisos de macOS (el paso que todos se saltan)

- Sistema → Seguridad y privacidad → Micrófono: permite Personal Jarvis.
- Accesibilidad: permite si vas a usar computer-use (abrir apps, clicks).
- Automatización / Accesibilidad: macOS 12 pregunta la primera vez. Si dices que no, el mouse “no hace nada” y parece un bug.
- Prueba de audio: di “Loona” y luego “qué hora es”. Si no despierta, baja el umbral o acércate al mic.

PASO B5 — Primeras frases que deben funcionar

Dices	Debe pasar
“Loona, recuerda: Chuy trabaja en Monterrey y el proyecto P1 es Likinya.”	Queda en la Knowledge Wiki (Markdown en disco).
“Ábreme el workspace /Users/imac/loona en el Finder.”	Computer-use: se ve un borde en pantalla mientras opera.
“Dicta esto” + hold key	El texto limpio cae en el campo con foco.
“Investiga X y déjame un reporte en Outputs.”	Misión aislada + critic. Tarda minutos. No es silencio: debe decir qué está haciendo.

PASO B6 — Cómo convive con OpenClaw (sin pelearse)

- Personal Jarvis = cuerpo (mic, voz, escritorio).
- OpenClaw = sistema nervioso 24/7 (Telegram, cron, skills cuando el iMac está idle).
- No conectes los dos al mismo bot de Telegram el día 1.
- Si quieres que la voz dispare al Gateway, hazlo después con un tool/MCP explícito. No lo improvises.
- La wiki de Personal Jarvis y SOUL.md de LOONA deben decir lo mismo sobre vetos (no política, no publish sin OK, no keys).

8. Paso a paso C — voz local (Hugging Face speech-to-speech)

Haz esto solo cuando A y B ya contestan. Es el camino para hablarle a LOONA sin mandar audio a un proveedor, o para tener latencia tipo “Realtime” con piezas open source.

Repo: <https://github.com/huggingface/speech-to-speech> · Apache-2.0 · `pip install speech-to-speech` · Python 3.10+

Qué es

Un pipeline modular: Silero VAD → STT (Parakeet TDT por default) → LLM (cualquier OpenAI-compatible, incluido llama.cpp / vLLM / Ollama vía proxy) → TTS (Qwen3-TTS). Expone `ws://localhost:8765/v1/realtime`, compatible con clientes OpenAI Realtime. En este iMac Intel, `mlx-audio` no aplica; usa los backends CPU/GGML.

PASO C1 — Instala el paquete

```
python3 -m venv /Users/imac/loona/.venv-s2s
source /Users/imac/loona/.venv-s2s/bin/activate
pip install -U pip
pip install speech-to-speech
```

PASO C2 — Modo híbrido (recomendado en Intel)

STT y TTS locales, cerebro cloud. Es el mejor tradeoff en un i5-6500:

```
export OPENAI_API_KEY=... # o HF_TOKEN / GEMINI / etc.
speech-to-speech local \
--stt parakeet-tdt \
--llm_backend responses-api \
--model_name gpt-4o-mini \
--tts qwen3
```

PASO C3 — Modo 100% local (lento en esta máquina)

```
# Terminal 1 — sirve un modelo chico
ollama serve
ollama pull qwen2.5:3b-instruct

# Si usas llama.cpp en vez de Ollama:
# llama-server -hf ggml-org/gemma-4-E4B-it-GGUF -np 2 -c 8192

# Terminal 2
speech-to-speech local \
--stt parakeet-tdt \
--llm_backend chat-completions \
--model_name qwen2.5:3b-instruct \
--responses_api_base_url http://127.0.0.1:11434/v1 \
--responses_api_key ollama \
--tts qwen3
```

La primera corrida descarga checkpoints (varios GB). Hazla con internet. Después puedes intentar HF_HUB_OFFLINE=1. Si la latencia supera ~8–10 s por turno, no es que esté roto: es el i5. Vuelve al modo híbrido.

9. Identidad, memoria y guardrails de LOONA

Esto es lo que separa un clone de Jarvis de un agente tuyo. Cópialo a `/Users/imac/loona/identity/SOUL.md` y conéctalo como instrucciones de sistema en OpenClaw y como wiki seed en Personal Jarvis.

Texto canónico de identidad (pegar tal cual y editar lo marcado)

```
# SOUL.md — LOONA
Eres LOONA, asistente personal de Chuy (abogado-constructor de sistemas de IA).
Zona: Monterrey (América/Monterrey). Idioma: español claro, skills/código en EN si aplica.

## Qué eres
- Sistema de 5 capas: identidad, memoria, tools, triggers, guardrails.
- Haces trabajo. No narres que “podrías” hacerlo.
- Reporta siempre: DONE | BLOCKED | NEED_HUMAN | IN_PROGRESS + evidencia.

## Proyectos
- P1 Likinya (no lo toques salvo que Chuy lo pida en esa sesión).
- Este workspace es LOONA. No mezcles write-paths con likinya.

## Nunca
- Publicar a redes, pagar, o borrar sin NEED_HUMAN.
- Inventar URLs, posts live, o “ya quedó” sin proof.
- Guardar passwords en el repo. No leer .env en voz alta.
- Política, hard-sell, o clonar personas reales.
- Pedirle a nadie que te mande un DM con una palabra mágica.

## Cómo decides
- Free / ya pagado primero (Grok, Claude, Gemini, Ollama).
- Una herramienta por turno si basta. No dispaes 8 tools de adorno.
- Si hay duda de irreversibilidad: pregunta. Una pregunta corta > un desastre.
```

Memoria mínima (MEMORY.md)

- Owner: Chuy. Studio multi-agente (Grok lead, Claude browser, Codex docs, Agy media).
- Stack: herdr, Publora, Notion, Drive, Make, browser-use. Free-first.
- LOONA vive en `/Users/imac/loona`. Space herdr: LOONA.
- Hardware: iMac Intel 24 GB. No asumir Apple Silicon ni CUDA.
- Preferencia de canal: Telegram para móvil, voz solo cuando el iMac está delante.

Guardrails operativos (cópialos a OpenClaw + Personal Jarvis)

Acción	Política
Leer archivos del workspace LOONA	Permitido
Escribir docs / identity / notas	Permitido
Ejecutar comandos de solo lectura (status, list, doctor)	Permitido
Instalar paquetes, cambiar config de red, LaunchAgents	Preguntar
Computer-use (mouse/teclado) fuera del workspace	Preguntar
Publicar a IG/TT/X, mail masivo, WhatsApp a terceros	Bloquear salvo OK explícito
Leer o pronunciar API keys, .env, llavero	Bloquear
Autonomía 24/7 (cron)	Solo jobs en lista blanca

10. Primeras 24 horas — checklist de prueba

No sigas al Track C ni a Flightdeck hasta tener esta tabla en verde. Evidencia = captura, log o cita textual. “Creo que sí” no cuenta.

#	Prueba	Cómo se ve el verde
1	OpenClaw doctor + gateway status	Listening :18789, sin errores de auth
2	Mensaje en Control UI	LOONA responde con su nombre, no como “assistant”
3	Telegram pairing	Un hola desde el celular recibe respuesta
4	Memoria	Le dices un hecho, cierras, reabres, lo recuerda
5	Tool real	Crea o lee un archivo en /Users/imac/loona/docs y te cita el path
6	Wake word “Loona”	El orb/app despierta; no se activa con la tele
7	Voz ida y vuelta	Pregunta hablada → respuesta hablada < 8 s (cloud)
8	Guardrail	Le pides publicar o leer un .env y se niega / pide OK
9	Costo	Revisas el usage del provider. Nada se disparó en loop
10	Reboot	Reinicias el iMac: el daemon de OpenClaw vuelve solo

Orden del día (no improvisar)

- **Mañana (90 min):** A1–A5. Para cuando el dashboard conteste.
- **Mediodía (45 min):** A6 Telegram + una skill/tool real.
- **Tarde (60 min):** B1–B5 voz. Si el installer pelea con Python 3.14, instala 3.12 y reintenta.
- **Noche (20 min):** SOUL.md + checklist 1–8. Escribe en BANDA_CEREBRO_LOONA.md qué faltó.
- **Día 2:** un cron útil. Luego speech-to-speech si la voz cloud te estorba.

11. Seguridad, costos y anti-patrones

Seguridad (de los propios READMEs, no de un guru de X)

- OpenClaw: trata todo DM como input no confiable. Pairing on. No expongas el Gateway a internet sin el exposure runbook.

- Tools corren en el host salvo que actives sandbox. Eso significa que un prompt inyectado puede intentar ejecutar cosas.
- Personal Jarvis: wake word es local; el audio solo sale si elegiste STT cloud y después de dirigirte a LOONA.
- Flightdeck (si algún día): LAN only, no port-forward. El token del HUD no es la API key de Hermes.
- Nunca dejes OPENAI_API_KEY en un LaunchAgent world-readable. umask y permisos 600.
- Un archivo SOUL.md no es un sandbox. Los guardrails tienen que vivir en config (ask/block), no solo en prosa.

Costos — cómo no despertar con una factura

Modo	Qué consume	Tope sano día 1
Solo OpenClaw + Gemini Flash / Grok ligero	Tokens de chat + 1–2 tools	USD 0–2 / día de uso normal
Personal Jarvis con STT/TTS cloud + mic abierto	Audio + realtime si lo encendiste	Cierra el mic. No dejes realtime 24/7
Misiones Claude/Codex cada frase	Workers caros	Reserva workers para “investiga / construye”
Ollama local	CPU/RAM, \$0 API	Úsalo de noche o para datos sensibles
ElevenLabs Flash	Créditos por caracter	Piper/edge-tts primero

Anti-patrones (vistos en X esta semana)

- Instalar OpenClaw + Flightdeck + Personal Jarvis + 3 “JARVIS.py” el mismo día.
- Pedirle a 4 agentes el mismo brief de “hazme Jarvis”.
- Creer que 385k stars = seguro de exponer WhatsApp al mundo.
- Usar el wake word “Jarvis” y que se dispare con YouTube.
- Declarar done porque “el HUD se ve increíble”. El HUD no es el producto. La tarea hecha sí.
- Seguir un hilo que termina en Discord de pago o DM. El código bueno ya está en GitHub.

12. Fuentes y repos canónicos

Fuente	URL	Fecha / nota
OpenClaw repo	https://github.com/openclaw/openclaw	MIT. Installer + Gateway + channels.
OpenClaw getting started	https://docs.openclaw.ai/start/getting-started	Onboard en ~5 min.
OpenClaw install	https://docs.openclaw.ai/install	Node 22.22.3+ / 24.15+ / 25.9+.
Personal Jarvis	https://github.com/PersonalJarvis/PersonalJarvis	v1.3.0, 10–12 ago 2026 en X.
HF speech-to-speech	https://github.com/huggingface/speech-to-speech	Citado por @Marco_Ramilli, 8 ago.
Flightdeck	https://github.com/MOVI85/flightdeck	macOS Apple Silicon. Fase futura.
mimiclaw (ESP32)	https://github.com/memovai/mimiclaw	@tom_doerr, 10 ago.
gbrain (memoria)	https://github.com/garrytan/gbrain	Garry Tan / YC. Fase 2.
5 capas de un assistant	https://x.com/fluixoo/status/2087488228165030159	12 ago 2026.
OpenClaw en X	https://x.com/sanjibxai/status/2087078552407814614	11 ago. 385k stars, MIT.

Siguiente movimiento concreto (hoy)

1) Quédate en el space herdr LOONA. 2) Corre el PASO A2 y A3 (instalar + onboard OpenClaw). 3) Cuando el dashboard conteste con el nombre LOONA, avísame y hacemos juntos Telegram + SOUL.md + el installer de Personal Jarvis. No clones Flightdeck todavía.

Documento generado el 12 de agosto de 2026 para el workspace LOONA. Los stars, versiones y flags de CLI se tomaron de los README oficiales ese día. Si un installer cambia, manda el repo, no este PDF. Este PDF es el mapa; el repo es la verdad.